

Setu — From SRS to Sprint-Ready Backlog

An AI reviewer that checks a draft SRS against the system we have already built — then drafts the backlog the delivery team works from.

1. Project Name

Setu — From SRS to Sprint-Ready Backlog

“Setu” means bridge — between the requirements the client agreed to and the backlog the team builds from.

2. Project Owner

Name: Anindo Dey

Employee PIN: 1808

Department / Unit: Software Engineering

Contact: anindo.dey@bracits.com | +8801324730001

3. Problem Statement

Our BAs typically prepare the SRS based on client discussions. While they understand the business requirements and intent well, they may not always have full visibility into the existing system behavior, database structure, or historical RCAs, which can sometimes lead to gaps in the requirements documentation:

- Requirements with no acceptance criteria
- Failure paths nobody thought to ask about
- Client requests that quietly contradict how the module already works
- Cross-module dependencies the BA had no way to see

These gaps often surface weeks later during development, QA, or even UAT—when fixing an ambiguity requires rework, clarification, and sometimes changes across multiple layers of the system. The cost of resolving an issue at this stage is significantly higher than identifying and addressing it during requirement sign-off, while the client is still available to clarify and negotiate the requirement.

Once the SRS is approved, it is independently translated by BAs, Developers, and QAs into features, tasks, and test cases. This creates room for interpretation gaps, which may only become visible after development or during testing—leading to rework and renewed requirement discussions.

There is also no consistent end-to-end traceability linking each requirement to its feature, development task, and test case. As a result, when a requirement changes, identifying its full impact becomes difficult and largely manual.

The obvious shortcut is closed to us: an SRS carries client business logic that cannot be pasted into a public AI tool.

The gap: requirement-review tools check a document against generic best practice. None of them check it against the system we have already built.

4. Proposed Solution

Setu sits between the BA’s draft SRS and the team’s backlog. Two phases, with one human gate between them.

Phase 1 — Gap analysis (before client sign-off)

1. The BA uploads the draft SRS (DOCX/PDF).
2. Setu parses it into discrete, numbered requirement statements.
3. Each statement is checked against an indexed knowledge base of **our own system**: source code, database schema, and historical RCA / defect records for the affected modules.
4. Output is a gap report — per requirement, with a severity and the evidence behind it (file, table, or ticket ID) — phrased as questions the BA can take straight into the next client meeting.
5. The BA accepts, edits, or dismisses each finding. Setu is advisory; the BA owns the document.

Human gate

Questions go back to the client; the SRS is revised, and the BA marks it approved. Nothing is generated and nothing is pushed anywhere until this happens.

Phase 2 — Role-aware drafts (after sign-off)

From the same analysis, Setu drafts what each role actually needs:

- **BA** — features and user stories with acceptance criteria in Given / When / Then form
- **Developer** — a task breakdown per story, naming the modules and tables each one touches
- **QA** — test scenarios, including the negative and failure paths flagged in Phase 1
- **Delivery lead / PMO** — a requirement → story → test traceability matrix

Every artifact is reviewed in-app and approved by a person before it can be exported (XLSX / DOCX) or pushed to Jira / Azure DevOps.

Hard deployment constraint: all inference runs on internally hosted models on our own infrastructure. Client business logic never leaves the network. This is a requirement of the design, not a preference.

5. Target Users

- **Business Analysts (primary)** — author the SRS, run the gap check, take the open questions back to the client.
- **Developers & Tech Leads** — get a task breakdown that already names the modules and tables involved.
- **QA Engineers** — get test scenarios covering the failure paths that would otherwise surface in UAT.
- **Project / Delivery Managers** — get traceability by default and a record of every gap raised and how it closed.
- **The client, indirectly** — gets asked the right questions while the requirement is still open.

6. Key Features

- **SRS ingestion & requirement extraction** — DOCX/PDF in, discrete numbered requirement statements out.
- **System knowledge base** — indexes source code, schema DDL and RCA history, per module.
- **Four gap checks** — missing acceptance criteria, unspecified failure paths, contradictions, hidden dependencies.
- **Evidence-linked findings** — every flag cites the file, table or RCA ticket it came from. No citation, no finding.
- **Client question sheet** — export the open gaps as a clean list to take into the client meeting.
- **Review & approval workflow** — accept / edit / dismiss each finding; explicit SRS sign-off gate.
- **Role-aware generation** — BA features + acceptance criteria, developer tasks, QA test scenarios.
- **Traceability matrix** — requirement → story → test, generated and kept in sync.
- **Export & push** — XLSX / DOCX download; optional Jira / Azure DevOps push on approval.
- **Audit trail** — what the AI proposed, what the human changed, who approved it and when.
- **On-premise only** — internally hosted models, zero external API calls.

7. AI Component

AI is used at five specific points:

1. **Requirement extraction** — an LLM segments SRS prose into atomic, testable requirement statements and classifies each as functional, non-functional, business rule or constraint.
2. **Retrieval (RAG)** — an embedding model indexes source code, schema DDL and RCA text into a vector store. Each requirement retrieves the slices of the existing system it actually relates to.
3. **Gap reasoning** — the LLM compares the requirement against retrieved system evidence and produces the four gap classes, each with a citation and severity. It answers only from retrieved evidence — uncited findings are suppressed.
4. **Drafting** — generation of acceptance criteria (Given/When/Then), developer task breakdowns and QA test scenarios from the approved requirement set.
5. **Traceability linking** — semantic matching of requirement ↔ story ↔ test case to build and maintain the matrix.

Where AI is deliberately not used: nothing is auto-approved, auto-committed or auto-pushed. The model proposes; a person disposes.

Model hosting: an open-weight instruct model (Llama 3.x / Qwen2.5-Coder class) served internally via vLLM or Ollama, a local embedding model (e.g. BGE-M3), and a self-hosted vector store. Zero egress.

8. Expected Outcome / Impact

- **Defects identified earlier, when they are easier and less costly to address.** Identifying a requirement gap during the SRS stage allows it to be clarified through discussion, while discovering the same gap during UAT may require rework or a formal change request.
- **Fewer post-sign-off change requests,** because the ambiguous points were raised before sign-off rather than after.
- **Backlog prep compressed** from days of manual re-cutting to reviewing a generated draft.
- **Consistent acceptance criteria across BAs** — the format stops depending on who wrote the document.
- **Institutional memory made usable.** The RCA archive becomes a check that runs on every new requirement.
- **Traceability by default,** which is useful evidence for our CMMI and ISO 9001 / 27001 commitments.
- **An AI gain we are actually allowed to take,** because client business logic stays inside our network.

9. Data / Resources Required

Data

- Historical and draft SRS / BRD documents (DOCX, PDF) for indexing and evaluation
- Source code repositories — read-only access for one or two pilot products
- Database schema — DDL export or data dictionary per pilot module
- RCA reports, defect and incident history (Jira / ticketing export)
- **A small labelled evaluation set:** past SRSs paired with the issues actually raised later — the one dataset we cannot measure without.

Systems & APIs

- Git (GitHub / GitLab / Bitbucket) read API
- Jira or Azure DevOps REST API — read for RCA history, write for optional story push
- Internal SSO / LDAP for authentication and role mapping
- File share or object storage for uploaded documents

Infrastructure

- Internal GPU server for LLM inference (vLLM / Ollama)
- Vector database (pgvector or Qdrant) + PostgreSQL for application data
- Web application stack and a background job queue for indexing

People & approvals

- One BA and one QA lead, a few hours a week, to validate findings — accuracy cannot be measured without them
- IT security sign-off for read access to repositories and schema

10. Success Criteria

The solution works if it clears these:

- **Recall on known gaps** — on past SRSs, Setu surfaces $\geq 60\%$ of the issues actually raised later in dev, QA or UAT.
- **Signal quality** — at least 70% of flags are marked “valid, worth asking the client” by a reviewing BA. Low precision kills adoption faster than low recall does.
- **Grounding** — 100% of findings carry a citation to a real file, table or ticket. Zero uncited findings reach the BA.
- **Time to first backlog** — stories, criteria and test scenarios for a medium module approved in under 30 minutes of BA time.
- **Traceability coverage** — 100% of approved requirements mapped to at least one story and one test scenario.
- **Security** — zero outbound calls to external inference APIs, demonstrable in network logs.
- **Adoption signal** — a BA runs a second document through it without being asked to.

Demo target for CodeSprint: one anonymised SRS, one indexed module, end to end — gap report → approval → backlog + matrix.

Why Should You Build This?

Build the project that can prevent requirement gaps before a single line of code is written.

Every team has seen defects that started with requirements that nobody challenged. **Setu tackles these gaps at the source—before they become code, rework, or UAT issues.**

It brings AI into real delivery work by securely using context from our **codebase, database schema, and RCA history** within our network.

The scope is focused and achievable: **RAG + four gap checks + a review interface**, with immediate validation against real projects.

This isn't AI looking for a problem—it solves one we already have. If successful, Setu can become part of how BRAC IT writes better requirements and delivers with greater confidence.

Choose Setu to build something technically exciting, immediately useful, and capable of creating lasting impact beyond CodeSprint.